

json-rpc



in Quarkus

json-rpc spec



Protocol

A stateless, lightweight RPC protocol encoded in JSON, designed for simple and reliable communication.



Transports

Transport-agnostic architecture. Works seamlessly over HTTP, WebSocket, TCP, stdio, and other protocols.



Timeline

2005 — Version 1.0 specification released.

2010 — Version 2.0 standard established.

<https://www.jsonrpc.org/specification>

json-rpc request / response

// Request

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "hello",  
  "params": {"name": "World"}  
}
```

// Response

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": "Hello World"  
}
```

json-rpc key concepts

Notifications

Requests without an id. The server must not reply (fire-and-forget style).

Batch Requests

Send an array of requests and receive a corresponding array of responses.

Params

Parameter structures can be by-name (using an object) or by-position (using an array).

Error Object

Contains {code, message, data} with standard codes:

- 32700 Parse error
- 32600 Invalid request
- 32601 Method not found
- 32602 Invalid params
- 32603 Internal error

Notable adopters



Ethereum nodes



Bitcoin core



**Language Server
Protocol (LSP)**



**Model Context Protocol
(MCP)**



KDE Plasma



Kodi (XBMC)

Dev UI

Quarkus Dev UI (v2, Quarkus 3+) replaced HTTP GET/POST with **JSON-RPC 2.0 over WebSocket** for real-time, bidirectional communication.



Browser Client

Lit Web Components + single persistent WebSocket



Backend Services

Plain CDI beans with public methods, registered via a build step

Dev UI

Backend Service Registration

```
@BuildStep
JsonRPCProvidersBuildItem register() {
    return new JsonRPCProvidersBuildItem(MyJsonRPCService.class);
}
```

Browser Client (JavaScript & Streaming)

```
jsonRpc = new JsonRpc(this);
this.jsonRpc.getStatus().then(r => { /* use r.result */ });

// Streaming:
this._observer = this.jsonRpc.streamLogs()
    .onNext(r => { /* handle item */ });
```

Dev UI



Communication Modes

Runtime classpath

(CDI beans)

Deployment classpath

(BuildTimeActionBuildItem)

Recorded values

(RuntimeValue)



Internal Infrastructure

Dev Mode Only: Only runs in dev mode.

Tightly Coupled: Deeply coupled to the build-item system.

Internal API: Not intended to be user-facing.

Limitations: No support for by-position params or batch operations.

quarkus-json-rpc

The general-purpose,
production-ready JSON-RPC
extension for Quarkus.

Expose any CDI bean as a JSON-RPC service
instantly with just one annotation.

WebSocket Endpoint

`ws://localhost:8080/json-rpc`

Call methods directly via `GreetingService#hello`

```
@JsonRPCApi
public class GreetingService {
    public String hello(String name) {
        return "Hello " + name;
    }
}
```

GitHub Repository: <https://github.com/quarkiverse/quarkus-json-rpc>

quarkus-json-rpc



Core & Discovery

Build-time discovery:

Jandex-powered, GraalVM native ready

Service Discovery: OpenRPC at `/json-rpc/openrpc.json`

CDI Lifecycle: Events via CDI (JsonRPCConnected, etc.)

Custom Paths: Set per class via `@JsonRPCApi`



Execution & Streaming

Execution Modes: plain, `@Blocking`, `@NonBlocking`, `@RunOnVirtualThread`

Asynchronous: Native support for `Uni<T>` & `Multi<T>`

Subscriptions: `subscribe` → ID → receive notifications → `unsubscribe`

Broadcasting: Push to all clients or target a session



Security & Integration

Security: `@RolesAllowed`, `@Authenticated`, bearer tokens via `WebSocket`

Client Generation: Automated typed JavaScript proxy client

Observability: Built-in Health checks & Micrometer metrics

Timeouts: Global or method-specific `@Timeout`

MCP: json-rpc over SSE/HTTP/stdio

Model Context Protocol

An open standard developed by [Anthropic](#) (released November 2024) [cite: 11].

It standardizes how AI assistants securely connect to external tools, APIs, and diverse data sources [cite: 11].

The "USB-C for AI" Analogy [cite: 11]

Instead of building custom, fragmented integrations between every AI application and every unique data source...

Build one MCP server, and any MCP-compatible client can immediately plug into it [cite: 11].

Growing Client Ecosystem

These popular environments and AI clients support MCP natively or via extensions:

- ✓ Claude Desktop & Claude Code
- ✓ VS Code (Copilot)
- ✓ Cursor & Windsurf
- ✓ JetBrains IDEs

...and many more

MCP: json-rpc over SSE/HTTP/stdio



Tools

Executable functions that the LLM can actively invoke during interaction.

EXAMPLE

"Get weather for a city"



Resources

Read-only data sources made safely available to the model as context.

EXAMPLES

Files, DB schemas, API docs



Prompts

Reusable structured templates and interaction workflows for the LLM.

EXAMPLE

"Summarize this code"

`tools/list`, `tools/call`, `resources/list`, `resources/read`, `prompts/list`, `prompts/get`

MCP: quarkus-mcp-server

Quarkus developers expose MCP capabilities with annotations:

```
@Tool(description = "Get weather alerts...")
String getAlerts(
    @ToolArg(description = "Two-letter...")
    String state) {
    return weatherClient.getAlerts(state);
}
```



Build-time Scanning

Jandex-powered discovery for `@Tool`, `@Resource`, & `@Prompt`.



Auto-Registration

Capabilities automatically registered in the MCP `initialize` response.



Runtime Dispatch

Incoming JSON-RPC requests efficiently dispatched to annotated methods.



Multi-Transport Support

Seamless operation over stdio, Streamable HTTP, SSE, and WebSocket.

Dev MCP: json-rpc over HTTP



Built-in MCP Server

Quarkus automatically exposes a dedicated Model Context Protocol server during development.

DEV MODE PATH

`/q/dev-mcp`

Uses the same JSON-RPC services infrastructure from Dev UI that extensions already provide.



Protocol & Connection

Connect your favorite AI clients and agents directly to the local development session.

SUPPORTED PROTOCOL

MCP Streamable HTTP (2025-03-26)

CONNECTION ENDPOINT

`http://localhost:8080/q/dev-mcp`

Dev MCP: json-rpc over HTTP

```
public class MyExtensionRPCService {  
    // Dev UI only -- no description  
    public JsonObject getInternalStats() { ... }  
  
    // Dev UI + Dev MCP -- visible to AI agents  
    @JsonRpcDescription("Update a logger's level in this application")  
    public JsonObject updateLogLevel(  
        @JsonRpcDescription("The logger name") String loggerName,  
        @JsonRpcDescription("The new log level") String levelValue) { ... }  
}
```